

Partial Pooling Across Hosts: A Multilevel Model for Airbnb Listing Prices

Statistical Rethinking, Chapter 13 — Worked Exercise

TalRamot46*

July 29, 2026

Abstract

This essay applies the varying-intercepts multilevel model of Chapter 13 of *Statistical Rethinking* to [data set]. The outcome of interest is [outcome], grouped by [cluster variable]. We fit [model] using Hamiltonian Monte Carlo and compare the resulting cluster-level estimates to their no-pooling and complete-pooling counterparts. We find [headline result], which illustrates [the shrinkage / regularisation lesson].

Contents

1	Introduction and Research Question	1
2	Data	1
2.1	Source and Variables	1
2.2	Preprocessing	2
3	The Model	3
3.1	Statistical Specification	3
3.2	Prior Justification	3
3.3	Comparison Models	3
4	Fitting and Diagnostics	3
4.1	Diagnostics	4
5	Results	4
5.1	Population-Level Parameters	4
5.2	Variance Decomposition	4
5.3	Host-Level Intercepts	4
6	Shrinkage and Partial Pooling	4
7	Limitations	4
8	Conclusion	4
A	R Code	4
A.1	Multilevel Model via <code>rethinking::ulam</code>	4
A.2	Equivalent Model via <code>brms</code>	6
A.3	Reproducibility	7

*Replace with your full name, ID, and course details.

1 Introduction and Research Question

Chapter 13 of *Statistical Rethinking* introduces multilevel (hierarchical) models as a principled compromise between two extremes: *complete pooling*, which ignores cluster identity entirely, and *no pooling*, which estimates each cluster in isolation. [Extend this motivation to your own setting.]

The data used here are [describe source]. Each row is a single listing, but listings are nested within hosts: a single host may operate many listings, and those listings plausibly share a common pricing strategy. Treating the rows as independent would therefore overstate the effective sample size and understate uncertainty.

Our research question is:

[State it as a question about $\bar{\alpha}$, σ_{host} , or the ratio between them.]

The remainder of the essay proceeds as follows. Section 2 describes the data and preprocessing. Section 3 states the statistical model and justifies each prior. Section 4 reports the fitting procedure and diagnostics. Section 5 presents the posterior. Section 6 examines shrinkage directly, and Section 8 concludes.

2 Data

2.1 Source and Variables

The raw file `listings.csv` contains $[N_{\text{raw}}]$ listings and 91 columns. The variables used in this analysis are summarised in Table 1.

Table 1: Variables used in the analysis.

Variable	Type	Description
<code>host_id</code>	integer	Identifier of the host; defines the cluster.
<code>price</code>	string	Nightly price as scraped, e.g. \$120.00.
<code>review_scores_rating</code>	real	Mean guest rating on a 0–5 scale.
<code>number_of_reviews</code>	integer	Count of reviews; used as a filter.
<code>price_clean</code>	real	<i>Derived.</i> <code>price</code> parsed to a number.
<code>log_price</code>	real	<i>Derived.</i> $\log(\text{price_clean})$; the outcome.
<code>host_idx</code>	integer	<i>Derived.</i> Contiguous index $1, \dots, J$ over hosts.

2.2 Preprocessing

Three preprocessing choices deserve comment. First, listings with no reviews are dropped, since [reason]. Second, price is modelled on the log scale because [reason]; this also guarantees that the Gaussian likelihood places no mass on negative prices once exponentiated. Third, the data are subsampled to $n = 2000$ rows to keep sampling times manageable, at the cost of [reason].

Cluster structure. After filtering, there are $J = [J]$ distinct hosts across $n = 2000$ listings, a mean of $[n/J]$ listings per host. [Report the distribution — how many hosts contribute exactly one listing? Those are the clusters that shrink the most.]

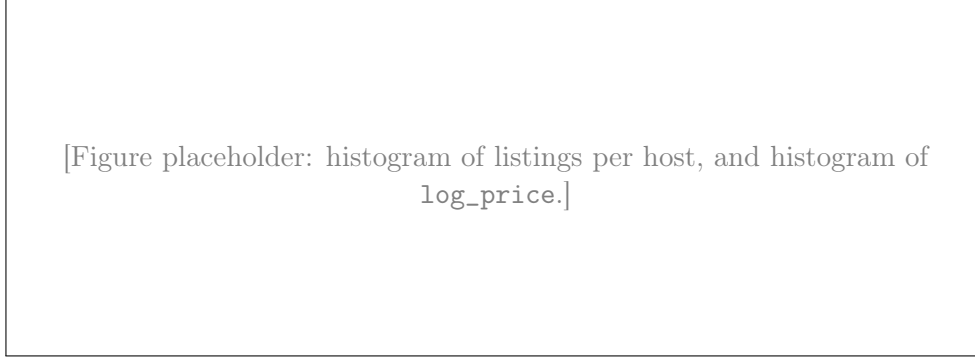


Figure 1: Distribution of listings per host (left) and of `log_price` (right).

3 The Model

3.1 Statistical Specification

Let y_i denote `log_price` for listing $i = 1, \dots, n$, and let $j[i] \in \{1, \dots, J\}$ denote the host of listing i . The varying-intercepts model is

$$y_i \sim \text{Normal}(\mu_i, \sigma) \tag{1}$$

$$\mu_i = \alpha_{j[i]} \tag{2}$$

$$\alpha_j \sim \text{Normal}(\bar{\alpha}, \sigma_{\text{host}}) \quad [\text{adaptive prior}] \tag{3}$$

$$\bar{\alpha} \sim \text{Normal}(4.5, 1) \quad [\text{hyper-prior}] \tag{4}$$

$$\sigma_{\text{host}} \sim \text{Exponential}(1) \quad [\text{hyper-prior}] \tag{5}$$

$$\sigma \sim \text{Exponential}(1). \tag{6}$$

The third line is what makes this a multilevel model. The prior for each host intercept α_j is not fixed in advance but is itself estimated from the data through $\bar{\alpha}$ and σ_{host} . Hosts therefore *learn from one another*: a host with a single listing is pulled towards the population mean $\bar{\alpha}$, whereas a host with many listings largely retains its own estimate.

3.2 Prior Justification

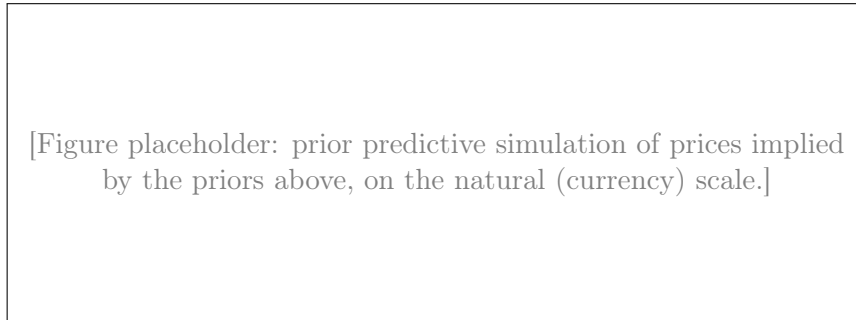


Figure 2: Prior predictive check. Priors should permit plausible prices without concentrating mass on absurd ones.

3.3 Comparison Models

- **Complete pooling:** $\mu_i = \alpha$, one intercept for all listings.

- **No pooling:** $\mu_i = \alpha_{j[i]}$ with $\alpha_j \sim \text{Normal}(4.5, 1)$ fixed — each host estimated independently.
- **Partial pooling:** the model above.

Compare them with `compare(m1, m2, m3, func=WAIC)` and discuss the effective number of parameters, which is the clearest quantitative signature of regularisation.

4 Fitting and Diagnostics

The model was fit with `ulam()`, which compiles the formula to Stan and samples via Hamiltonian Monte Carlo: 4 chains, 2000 iterations each (half warm-up). A `brms` implementation of the same structure is included for comparison.

4.1 Diagnostics

Table 2: Convergence diagnostics for the population-level parameters.

Parameter	Mean	SD	ESS	$\hat{R}$
$\bar{\alpha}$	—	—	—	—
σ_{host}	—	—	—	—
σ	—	—	—	—

5 Results

5.1 Population-Level Parameters

5.2 Variance Decomposition

$$\text{ICC} = \frac{\sigma_{\text{host}}^2}{\sigma_{\text{host}}^2 + \sigma^2} = [\cdot] \quad (89\% \text{ CI} : [\cdot, \cdot]). \quad (7)$$

5.3 Host-Level Intercepts

[Figure placeholder: caterpillar plot of the α_j posteriors, ordered by mean, with $\bar{\alpha}$ as a horizontal reference line.]

Figure 3: Posterior means and 89% intervals for host intercepts α_j .

[Figure placeholder: no-pooling estimate vs. partially pooled estimate per host; point size $\propto$ listings per host; 45° line and $\bar{\alpha}$ shown.]

Figure 4: Shrinkage of host-level estimates towards the population mean.

6 Shrinkage and Partial Pooling

7 Limitations

8 Conclusion

A R Code

A.1 Multilevel Model via `rethinking::ulam`

```
1 library(tidyverse)
2 library(rethinking)
3
4 # 1. Load and clean data
5 df <- read_csv("listings.csv") %>%
6   # Select required columns (review_scores_rating exists in this detailed
7   # dataset)
8   select(host_id, price, review_scores_rating, number_of_reviews) %>%
9   filter(!is.na(price), !is.na(review_scores_rating), number_of_reviews > 0)
10   %>%
11   mutate(
12     # Clean price: remove '$' and ',' then parse to numeric
13     price_clean = parse_number(as.character(price)),
14     log_price   = log(price_clean)
15   ) %>%
16   filter(price_clean > 0) %>%
17   slice_sample(n = 2000)
18
19 # 2. Rethinking requires CLEAN integer index numbers (1, 2, 3, ... N)
20 df$host_idx <- as.integer(as.factor(df$host_id))
21
22 # 3. Create list for ulam
23 dat <- list(
24   log_price = df$log_price,
25   host_idx  = df$host_idx
26 )
27
28 # 4. Chapter 13 Multilevel Model Formula
29 f_multilevel <- alist(
30   log_price ~ dnorm(mu, sigma),
```

```

30 # Linear model: unique intercept 'a' per host
31 mu <- a[host_idx],
32
33 # Adaptive Prior (Varying Intercepts / Partial Pooling)
34 a[host_idx] ~ dnorm(a_bar, sigma_host),
35
36 # Hyper-priors
37 a_bar ~ dnorm(4.5, 1),
38 sigma_host ~ dexp(1),
39
40 # Residual variation
41 sigma ~ dexp(1)
42 )
43
44 # 5. Fit model via ulam (Stan HMC engine)
45 m_rethinking <- ulam(
46   f_multilevel,
47   data = dat,
48   chains = 4,
49   cores = 4,
50   iter = 2000
51 )
52
53 # 6. View estimates (depth = 2 prints varying host intercepts alongside a_bar
54   and sigma)
55 sink("precis_output.txt")
56 precis(m_rethinking, depth = 2) # or depth = 1
57 sink()

```

Listing 1: rethinking_code.R — varying intercepts on log_price by host.

A.2 Equivalent Model via brms

```

1 library(tidyverse)
2 library(brms)
3
4 # 1. Load and clean data
5 # Make sure "listings.csv" is in your working directory, or specify the full
6   path
7 df <- read_csv("listings.csv") %>%
8   select(host_profile_id, review_scores_rating, number_of_reviews) %>%
9   filter(!is.na(review_scores_rating), number_of_reviews > 0) %>%
10  # Ensure host_profile_id is treated as a categorical factor for varying
11    intercepts
12  mutate(host_profile_id = as.factor(host_profile_id)) %>%
13  # Filter down to top N rows to keep sampling quick
14  slice_sample(n = 2000)
15
16 # 2. Set Bayesian Priors
17 # Chapter 13 focuses on partial pooling via varying intercepts (1 | host_id)
18 priors <- c(
19   prior(normal(4.5, 0.5), class = Intercept), # Prior on average rating
20   prior(exponential(1), class = sd),           # Prior on standard deviation
21   # across hosts
22   prior(exponential(1), class = sigma)        # Prior on residual variation
23   # within hosts
24 )
25
26 # 3. Fit Varying Intercept Model via brms (uses Stan backend under the hood)
27 m_partial_pooling <- brm(
28   formula = review_scores_rating ~ 1 + (1 | host_profile_id),
29   data = df,

```

```

26 family = gaussian(),
27 prior = priors,
28 chains = 4,
29 cores = 4,
30 iter = 2000,
31 warmup = 1000
32 )
33
34 # 4. View Model Summary
35 summary(m_partial_pooling)

```

Listing 2: code.R — the same varying-intercepts structure in **brms** syntax.

A.3 Reproducibility

References

- [1] McElreath, R. (2020). *Statistical Rethinking: A Bayesian Course with Examples in R and Stan*, 2nd ed. CRC Press. Chapter 13.
- [2] Bürkner, P.-C. (2017). brms: An R Package for Bayesian Multilevel Models Using Stan. *Journal of Statistical Software*, 80(1), 1–28.
- [3] Stan Development Team (2024). *Stan Modeling Language Users Guide and Reference Manual*.
- [4] Inside Airbnb. <http://insideairbnb.com/get-the-data/>. Accessed [date].